

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

In today's educational environment, maintaining accurate and up-to-date information about students after graduation has become essential for institutions. Traditional methods of collecting and managing alumni data are often manual, time-consuming, and lack proper organization. This leads to difficulties in tracking student progress and accessing relevant information when needed.

The Student Progression Management System is a web-based application designed to provide a centralized platform for managing and maintaining student progression details. The system enables students to update their personal and professional information, including their current status such as employment, higher studies, business, or other activities. It also allows institutions to store and organize this information in a structured and efficient manner, ensuring easy access and better data management.

By utilizing technologies such as **HTML**, **CSS**, and **JavaScript** for the frontend, and **PHP** with **MySQL** for the backend, the system ensures a user-friendly interface and reliable data handling. It helps reduce manual effort, improves data accuracy, and supports effective monitoring of student progression. Overall, the system provides a streamlined digital solution for managing alumni records and enhancing institutional efficiency.

1.1 Feasibility Study

Feasibility study is an important step in system development which determines whether the proposed system is practical and beneficial. It helps in analyzing the system from different aspects such as technical, operational, and economic feasibility.

The study focuses on three main aspects:

- Technical Feasibility
- Operational Feasibility
- Economic Feasibility

1.2.1 Technical Feasibility

Technical feasibility evaluates whether the required technology, tools, and resources are available to develop and run the system efficiently.

- **Software Requirements:** The frontend is developed using HTML, CSS, and JavaScript, while PHP is used for backend processing. MySQL is used as the database to store student details. Tools like XAMPP or WAMP server are used to run the system locally.
- **Hardware Requirements:** The system can run on standard computer systems with minimum requirements such as an Intel i3 processor or above, 4 GB RAM, and sufficient storage space. Basic input and output devices like keyboard, mouse, and monitor are required.
- **Server Requirements:** The application requires a server environment such as XAMPP or WAMP for local execution. It uses Apache as the web server and MySQL as the database server. For online usage, a hosting server with internet connectivity is required.

1.2.2 Operational Feasibility

Operational feasibility checks whether the system is user-friendly and can be effectively used in real-time.

- **User Interface:** The system provides a simple and clean interface with easy navigation. The landing page includes options like Home, Student Login, and Admin Login, making it easy for users to access features.
- **Ease of Use:** Students can easily log in using their register number and update their details without any technical knowledge. The admin can monitor and manage data through a dashboard.
- **Maintenance and Support:** The system is easy to maintain as it uses simple technologies. Errors can be fixed quickly, and updates can be done without affecting the overall system.

1.2.3 Economic Feasibility

Economic feasibility evaluates the cost-effectiveness of the system.

- **Cost Analysis:** The system requires minimal investment as it uses open-source technologies. There is no need for expensive software or licenses.
- **Development Cost:** Development cost is low because the tools used (HTML, CSS, JavaScript, PHP, MySQL) are free. The project can be developed using existing resources like personal computers.
- **Maintenance Cost:** Maintenance cost is also low. Only minimal expenses are required for hosting, updates, and system support. The system reduces paperwork and manual work, saving time and money.

1.2.3 Legal Feasibility

Legal feasibility refers to whether the proposed system complies with all applicable laws, regulations, and policies.

- The system primarily deals with student information such as academic records, personal details, and progress data.
- The system ensures that sensitive information is accessed only by authorized users through secure login and role-based access control.

Additionally, the system maintains accurate records and avoids any kind of data manipulation, ensuring transparency and reliability. It can also be updated in the future to comply with any new legal requirements or institutional policies.

CHAPTER 2

SYSTEM ANALYSIS

2.1 Aim

The main aim of the **Student Progression Management System** is to develop a web-based application to track and manage the career progression of students after graduation. The system helps institutions collect, update, and maintain accurate records of students' current status such as **working, higher studies, business**, or others. It provides an easy platform for students to update their details regularly and for the admin to monitor and analyze the data efficiently.

2.2 Existing System

The existing system for managing student progression is mostly manual and lacks efficiency. Student details are collected through offline methods such as paper forms, phone calls, or emails, making the process slow and difficult to manage.

Key Points:

- **Manual Process:** Student data is collected and maintained manually, which increases workload and chances of errors.
- **Time Consuming:** Collecting and updating student information takes a lot of time and effort.
- **Lack of Centralization:** There is no single platform to store and manage all student data in one place.
- **Outdated Information:** Student details are not updated regularly, leading to inaccurate records.
- **Data Loss Risk:** Paper-based records can be easily lost or damaged.

2.3 Proposed System

The proposed Student Progression Management system provides a centralized platform where students can log in and update their personal and professional details such as current status, contact information, and supporting documents.

Key Features of Proposed System

- **Student Login:** Students can securely log in using their register number and name. This ensures that only authorized users can access and update their details.
- **Profile Management:** Students can enter and update their personal information such as email, phone number, and address. This helps in maintaining accurate and up-to-date records.
- **Status-Based Data Entry:** The system allows students to select their current status such as working, higher studies, business, or others. Based on the selection, relevant details are collected, making the system flexible and organized.
- **History Tracking:** The system stores all previous updates with date and time. This helps in tracking the career growth and changes of students over a period of time.
- **Admin Dashboard:** The admin can view an overview of total students and category-wise distribution. This helps in analyzing how many students are working, studying, or engaged in other activities.

2.4 Modules

The Student Progression Management System is divided into several modules to ensure smooth functioning and better management of student data. Each module is designed to perform specific tasks efficiently.

2.4.1 Student Login Module

This module allows students to log in securely using their register number and name. It ensures that only authorized users can access their profile and update details.

Functionalities:

- Allows students to enter register number and name to access the system securely
- Verifies login credentials with database to ensure authorized access
- Prevents unauthorized users from accessing the system
- Maintains user session after successful login

Key Components:

- Login form for entering register number and name
- Input validation to check correct data format
- Authentication system to verify user identity

2.4.2 Student Profile Module

This module is used to collect and manage student personal details such as name, email, phone number, and address

Functionalities:

- Enables students to enter personal details such as name, email, and phone number
- Allows editing and updating of profile information whenever required
- Displays stored profile details for user reference
- Validates input data to maintain accuracy and consistency
- Stores all student information securely in the database
- Maintains updated and organized student records

Key Components:

- Profile form to collect student information
- Input fields for entering personal details
- Validation system to ensure correct data entry

2.4.3 Status Update Module

This module allows students to update their current status such as working, higher studies, business, or others.

Functionalities:

- Allows students to select current status such as working, higher studies, business, or
- Dynamically displays relevant fields based on selected status
- Collects detailed information like company name, salary, or college details

Key Components:

- Status selection option (dropdown or radio buttons)
- Dynamic form fields based on selected status
- Input validation for entered details

2.4.4 Document Upload Module

This module allows students to upload or provide links for documents such as ID card, offer letter, photo, and signature.

Functionalities:

- Allows students to upload documents such as ID card, photo, and signature
- Enables adding external links like Google Drive for document proof
- Stores document links and file references securely
- Validates file format and size before upload
- Provides option to view uploaded documents
- Maintains proper documentation for verification purposes

Key Components:

- File upload field to select documents
- Link input box for external document links
- Storage system to manage uploaded files
- Validation process for file type and size

2.4.5 History Tracking Module

This module maintains all previous updates of student data along with date and time

Functionalities:

- Records every update made by the student in the system
- Stores previous details as history with date and time
- Displays history records for tracking changes
- Helps in analyzing student career progression over time

Key Components:

- History table in database to store past records
- Timestamp feature to record date and time
- Database connection for storing and retrieving data
- Retrieval system to fetch previous updates

2.4.6 Admin Module

This module allows the admin to control and manage the entire system.

Functionalities:

- Allows admin to log in securely using username and password
- Provides access to view all student details in the system
- Enables monitoring of student updates and activities
- Allows management and control of system data

Key Components:

- Admin login form for authentication
- Dashboard to view system overview

This system helps solve that problem by providing an easy and efficient way to track student progression. The system allows students to update their profile regularly, and all previous records are stored as history with date and time. This helps in tracking the growth and changes in a student's career over time.

CHAPTER 3

SYSTEM SPECIFICATION

System Specification refers to the detailed description of the hardware and software requirements needed to develop and run the system. It defines the technical environment in which the system operates. It includes hardware requirements such as processor type, RAM, storage, and system configuration. It also includes software requirements such as programming languages, database systems, development tools, and operating systems used in the project.

3.1 HARDWARE REQUIREMENTS

The hardware requirements define the basic configuration needed to develop and run the Student Progression Management System effectively. The system does not require high-end hardware and can operate on standard computing devices.

3.1.1 Minimum Requirements

The system can function with the following minimum configuration:

- Operating System: Windows 7 or later
- Processor: Dual Core Processor
- RAM: 2 GB
- Hard Disk: 10 GB free space
- Monitor: Standard display
- Keyboard and Mouse: Standard input devices
- Browser: Google Chrome / Firefox

3.1.2 Recommended Specifications

For better performance, the following configuration is recommended:

- Operating System: Windows 10 / 11
- Processor: Intel i3 or above
- RAM: 8 GB or higher
- Hard Disk: 500 GB or higher
- Monitor: HD display

3.2 SOFTWARE REQUIREMENTS

The software requirements define the tools and technologies used for development and execution of the system.

- Frontend: HTML, CSS, JavaScript
- Backend: PHP
- Database: MySQL
- Server: XAMPP / WAMP
- IDE: Visual Studio Code
- Browser: Chrome / Edge / Firefox

3.3 SOFTWARE DESCRIPTION

3.3.1 HTML

HTML is used to design and structure the web pages of the Student Progression Management System. It is mainly used to create pages like student login, admin login, and profile update forms. HTML helps in organizing content such as text fields, buttons, and input forms. It provides the basic layout for entering and displaying student details. It ensures that all elements are properly arranged in the system.

3.3.2 CSS

CSS is used to improve the appearance of the web application. It is applied to design colours, fonts, spacing, and layout of the pages. CSS helps to make the system look attractive and user-friendly. It also ensures that the website is responsive and works properly on different devices. This improves the overall user experience of the system.

3.3.3 JAVASCRIPT

JavaScript is used to add interactivity to the system. It helps in validating form inputs such as checking empty fields and correct data format. JavaScript also provides dynamic features like showing or hiding fields based on status selection. It improves the user experience by making the system more interactive. It also reduces errors during data entry.

3.3.4 PHP

PHP is used as the backend language in this system. It handles server-side operations such as login authentication and data processing. PHP is responsible for storing and retrieving data from the database. It connects the frontend with the database and manages all system functions. It ensures smooth communication within the system.

3.3.5 MySQL

MySQL is used as the database to store all student information. It stores details such as personal data, status updates, and history records. MySQL helps in organizing data in tables for easy access and management. It supports data retrieval, insertion, and updating operations. It ensures data is stored securely and efficiently.

3.3.6 XAMPP SERVER

XAMPP is used as a local server to run the project. It provides an environment to execute PHP and MySQL together. It helps in testing the system before deployment. XAMPP includes Apache server and database support. It is easy to install and useful for development purposes.

3.3.7 VISUAL STUDIO CODE

Visual Studio Code is used as the code editor for developing the project. It supports multiple programming languages like HTML, CSS, JavaScript, and PHP. It provides features like syntax highlighting and auto-completion. It helps in writing and editing code efficiently. It improves the overall development process.

CHAPTER 4

SYSTEM DESIGN

System Design is the process of defining the architecture, structure, and workflow of a system before its actual development. It provides a clear blueprint of how the system will function and how different components will interact with each other. The main purpose of system design is to ensure that the system is well-organized, efficient, and easy to develop. It helps developers understand the overall structure and reduces errors during implementation. A good system design improves performance, scalability, and maintainability of the application.

4.1 ENTITY RELATIONSHIP DIAGRAM

An **Entity Relationship (ER) Diagram** is used to represent the structure of the database and the relationships between different entities in the system. It helps in understanding how data is stored, organized, and connected within the application. In the Student Progression Management System, the main entities include Student, Status, and Admin. The Student entity contains attributes such as **register number, name, email, phone number, and login credentials**. The Status entity stores details related to the student's current career progression, such as Working, Higher Studies, Business, or Others. Each status category contains specific attributes. For example, the Working category includes **company name, salary, and proof documents**, while the **Higher Studies** category includes course details and institution name. The Admin entity is responsible for managing and monitoring the entire system. It includes attributes such as admin ID, username, and password. The relationships between these entities are clearly defined. A student can update multiple status records over time, which creates a one-to-many relationship between Student and Status. The admin has access to all student records, which establishes control over the entire database. The ER diagram helps in designing a structured database and ensures efficient data storage and retrieval. It also reduces redundancy and improves data integrity within the system.

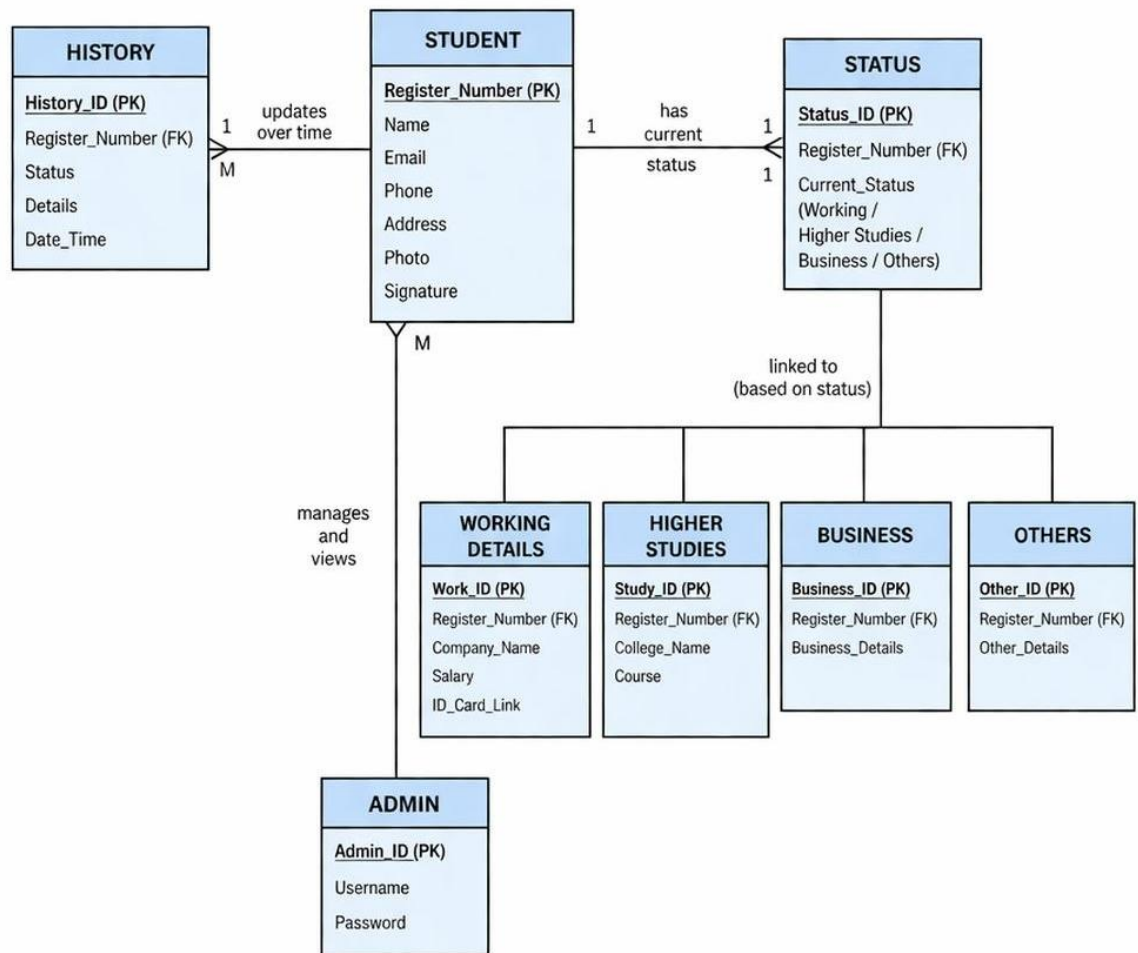


Fig 4.1 Entity Relationship Diagram

4.2 UML DIAGRAM

A **Unified Modelling Language (UML) diagram** is used to represent the structure and behaviour of the system in a visual format. In this system, the main classes include Student, Admin, and Status. The Student class consists of attributes such as register number, name, email, phone number, and login details. It also includes methods such as **login()**, **update Profile()**, and **upload Documents()**. The Admin class contains attributes like admin ID and password, with methods such as **view Data()**, **manage Records()**, and **generate Reports()**. The Status class stores information related to student progression. It includes attributes such as status type, details, and timestamp. Methods in this class include **add Status()** and **update Status()**. The UML diagram provides a clear blueprint of the system and helps in designing and implementing the application effectively.

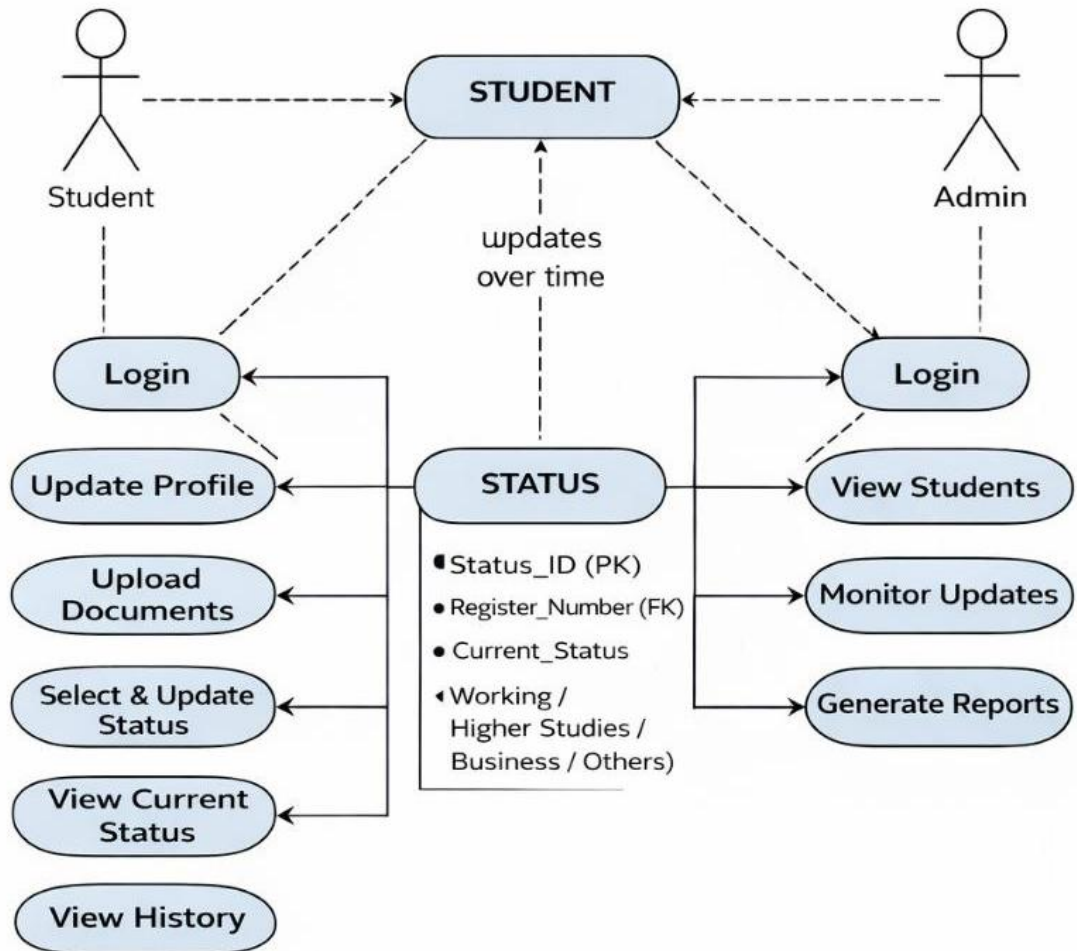
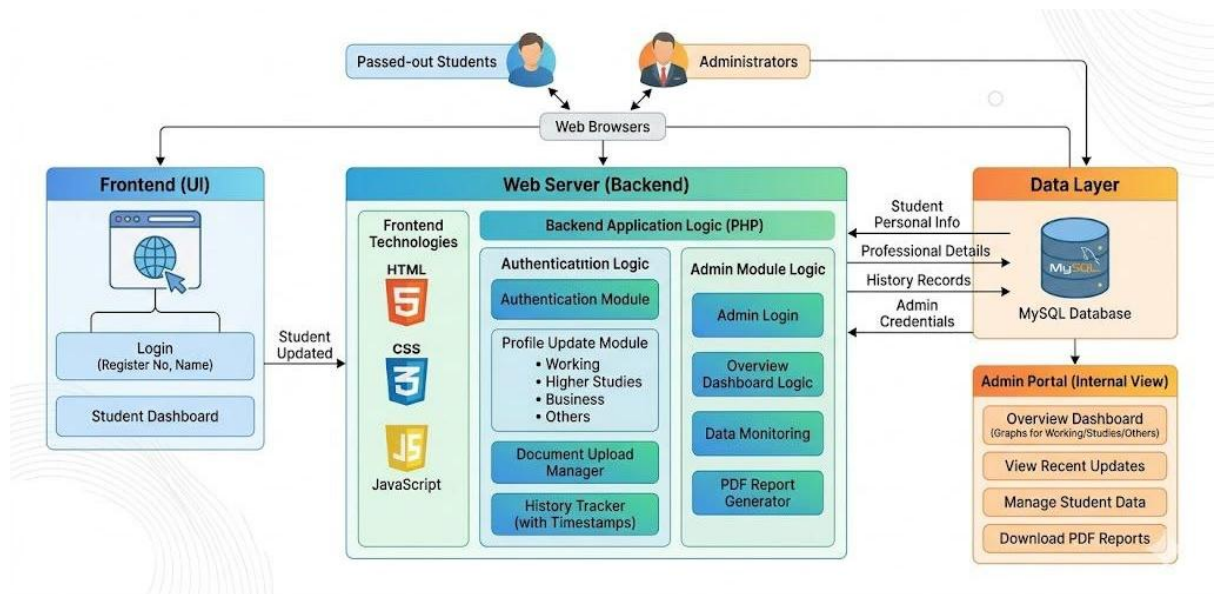


Fig 4.2 UML Diagram

4.3 SYSTEM ARCHITECTURE DIAGRAM

The **system architecture** of the **Student Progression Management System** represents the overall structure and interaction between different components of the application. It is designed as a web-based system where passed-out students and administrators access the system through web browsers. The frontend layer provides the user interface, developed using **HTML, CSS, and JavaScript**. It includes features such as student login and dashboard, allowing users to update their personal and professional details easily. The backend layer is implemented using **PHP**, which handles the application logic and processes user requests.

The backend includes modules such as authentication, profile update, document upload, and history tracking. It ensures that students can update their status such as working, higher studies, business, or others. The admin module allows administrators to log in, monitor student data, and generate reports. The data layer uses a MySQL database to store student information, professional details, history records, and admin credentials securely. Data flows between frontend and backend, and then to the database for storage and retrieval. The admin portal provides features such as dashboard overview, recent updates, data management, and report generation. Overall, this architecture ensures efficient data handling, secure access, and smooth interaction between users and the system.



i

Fig 4.3 SYSTEM ARCHITECTURE DIAGRAM

4.4 SEQUENCE DIAGRAM

The Sequence Diagram of the Student Progression Management System, which illustrates the interaction between the student, web application, admin, and database in a step-by-step manner. Initially, the student logs into the system using their register number and name through the web application. After successful login, the student enters personal profile details and selects their current status such as working, higher studies, business, or others. Based on the selected status, the student provides relevant information and uploads necessary documents like ID proof or certificates.

All the entered data is then processed by the web application and stored securely in the database. The student can also view and update their profile, and every update is recorded in the history section for future reference. On the other hand, the admin logs into the system to monitor student data. The admin can view the dashboard, analyze student status, and generate reports in PDF format. Overall, this diagram clearly shows how data flows between different components of the system and how each user interacts with the application over time.

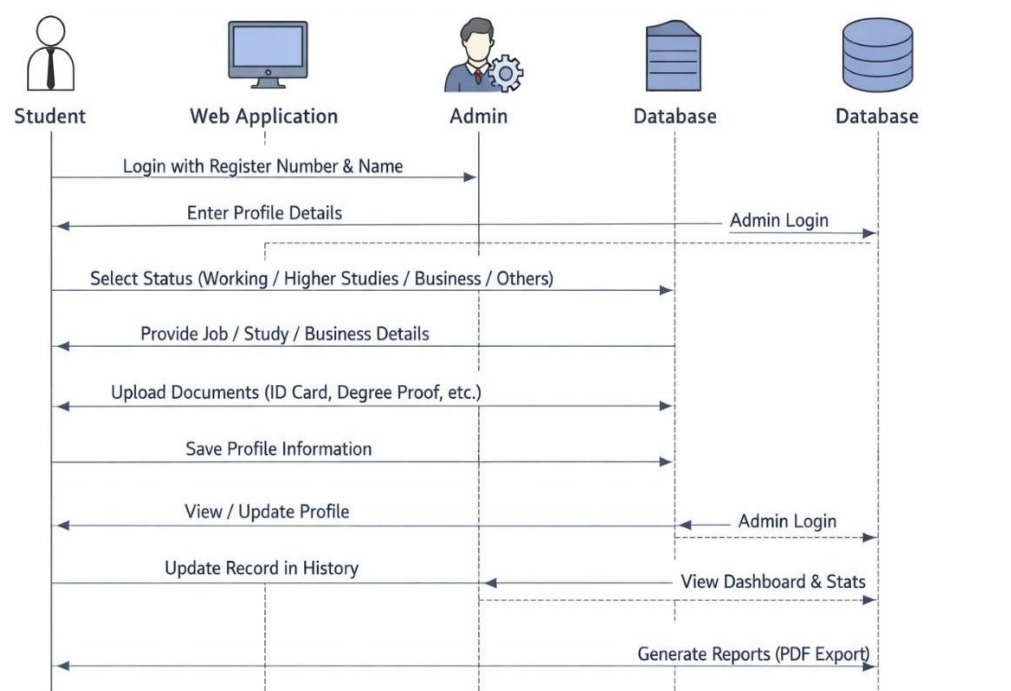


Fig 4.4 Sequence Diagram

4.5 STATE DIAGRAM

The State Diagram of the Student Progression Management System in a simple way. First, the process starts when the student logs into the system. After login, the student enters their details and updates their profile. Then the student selects their current status such as working, higher studies, business, or others. Next, based on the selected status, the student provides the required details. For example, if the student is working, they enter job details, and if they are studying, they enter study details. After that, the student uploads important documents like ID proof or certificates. Once all the details are entered, the system saves the profile information and moves to the next state.

Finally, the system stores the updated data in the history section. The admin can also generate reports from the stored data. This shows how the system moves to admin management state. Overall, the diagram explains how the system changes from one state to another based on student and admin actions in a simple and clear way.

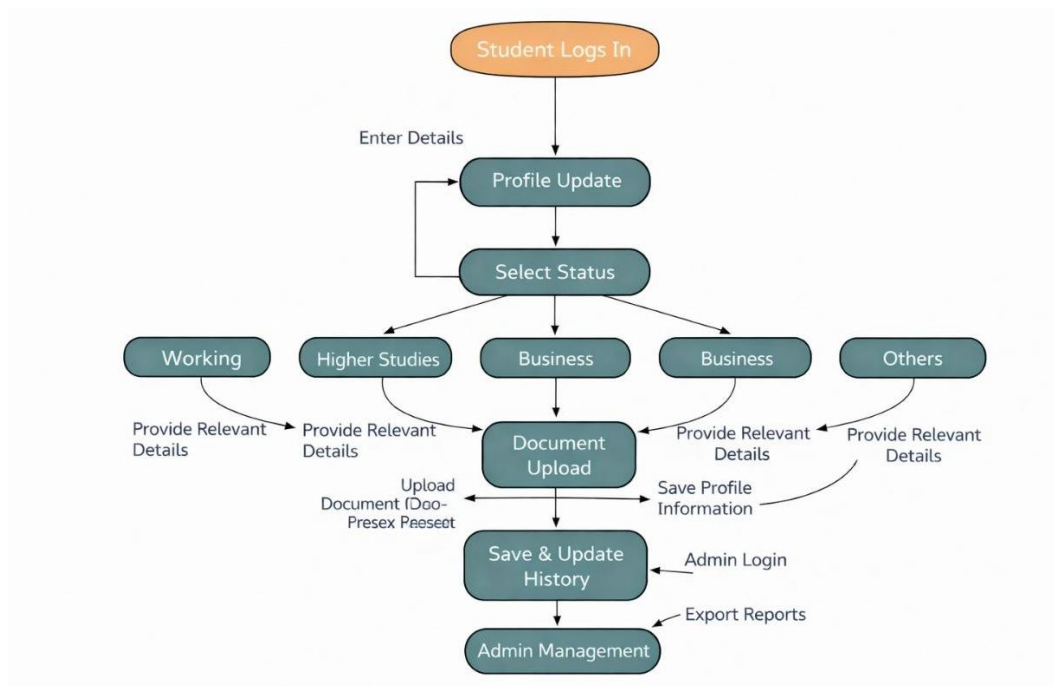


Fig 4.5 State Diagram

CHAPTER 5

TESTING

Testing is an essential phase in software development that ensures the system works correctly and meets user requirements. It involves executing the application with the aim of identifying errors and fixing them before final deployment. The main objective of testing is to verify that the system performs accurately, securely, and efficiently under different conditions. In the **Student Progression Management System**, testing plays an important role in validating various modules such as student login, profile update, status management, document upload, and admin monitoring. Proper testing ensures that students can update their details without errors and administrators can access and manage data effectively. Testing also helps in improving system reliability and user experience. Both frontend and backend functionalities are tested to ensure smooth communication between the user interface and database. This phase guarantees that the system replaces manual data handling with a secure and efficient digital solution.

5.1 UNIT TESTING

Unit testing involves testing individual components of the system separately to ensure that each module functions correctly. In the Student Progression Management System, unit testing is performed on modules such as login, data entry forms, and database operations.

- Each form is tested to ensure correct input validation, including checking required fields, email format, and proper data submission.
- The login module is tested to verify that only valid users can access the system. Backend functions such as storing and retrieving data from the database are also tested individually.

Unit testing helps in identifying errors at an early stage and ensures that each component works independently without issues.

5.2 INTEGRATION TESTING

Integration testing ensures that different modules of the system work together properly. It verifies the interaction between frontend and backend components. In this system, integration testing is performed to check whether the data entered by students is correctly processed by PHP scripts and stored in the MySQL database.

- Ensures that status updates, document uploads, and admin views are properly synchronized.
- The connection between login module, data update module, and database is tested to ensure smooth data flow. This testing ensures that the entire system functions as a unified application.

5.3 USER ACCEPTANCE TESTING (UAT)

User Acceptance Testing is conducted to verify that the system meets user expectations and works effectively in a real-world environment. It involves actual users testing the system for usability and functionality.

- The ease of navigation, clarity of forms, and overall user experience are evaluated. Administrators also test the system by viewing and managing student data.
- Feedback from users is used to improve the system, such as simplifying form fields and improving error messages.

In this project, students test the system by logging in and updating their details. This ensures that the final system is user-friendly and meets all requirements.

5.4 SYSTEM TESTING

- System testing evaluates the complete and integrated system to ensure that it functions correctly as a whole.
- It verifies that all modules work together without errors and meet the specified requirements.

5.4.1 FUNCTIONAL TESTING

- Functional testing ensures that all features of the system work as expected. It verifies student login, profile updates, status selection, and data storage.
- The system is tested to ensure that students can successfully submit their details and that the admin can view and manage the data without errors.

5.4.2 USER INTERFACE TESTING

- It ensures that web pages are well-structured, visually clear, and easy to navigate.
- All buttons, forms, and links are tested to ensure proper functionality.
- Error messages and confirmation alerts are also verified.

5.4.3 PERFORMANCE TESTING

- Performance testing ensures that the system works efficiently under different conditions.
- It checks system speed and response time when multiple users access the system.
- The system is tested to ensure fast data submission and retrieval without delays.

5.4.4 SECURITY TESTING

- Security testing ensures that the system protects user data and prevents unauthorized access.
- The system is tested for secure login, proper authentication, and safe data storage.

CHAPTER 6

SYSTEM IMPLEMENTATION

System implementation is the process of converting the system design into a working application. It involves developing, testing, and deploying the system to ensure that it meets the required objectives. This phase is important because it ensures that all modules function correctly and the system works efficiently in a real-world environment.

The implementation includes modules such as student login, profile update, status management, and admin dashboard. Students can log in using their register number and update their personal and professional details based on their current status such as working, higher studies, business, or others. The system also supports document uploads and maintains history records for future reference.

Proper validation and security measures are implemented to ensure data accuracy and prevent unauthorized access. The system is tested thoroughly to ensure that all functionalities work correctly without errors. Overall, the implementation phase ensures that the system is reliable, user-friendly, and capable of managing student progression data efficiently.

6.1 IMPLEMENTATION PLAN

The implementation of the Student Progression Management System was carried out through a structured and systematic approach to ensure efficiency, reliability, and ease of use. The implementation plan was designed to full fill all the functional requirements of the system while maintaining data security, accuracy, and user-friendliness. The major steps involved in the implementation process are as follows:

1. REQUIREMENT ANALYSIS

In this phase, the requirements of the system are identified and analyzed. The main objective is to track student progression after graduation. Functional requirements such as login, profile update, status selection, and admin monitoring are clearly defined. This phase helps in understanding user needs and system expectations.

2. SYSTEM DESIGN IMPLEMENTATION

The design of the system is converted into a working model during this phase. The database structure, user interface, and system workflow are implemented based on the design specifications. This phase acts as a bridge between design and development. It ensures that all design elements are properly translated into code. The system structure is implemented for smooth data flow. It also helps in reducing development errors.

3. FRONTEND DEVELOPMENT

The frontend of the system is developed using HTML, CSS, and JavaScript. It provides an interactive and user-friendly interface for students and administrators to access and use the system easily. All web pages are designed with proper layout and navigation. User input forms are created for data entry and updates. The interface ensures ease of use and clarity. It also improves user experience with responsive design.

4. BACKEND DEVELOPMENT

The backend is developed using PHP, which handles server-side processing. It manages user authentication, data submission, and communication with the database. It ensures that all operations are performed correctly. The backend processes user requests efficiently. It connects frontend and database smoothly. It also ensures secure handling of data transactions.

5. DATABASE IMPLEMENTATION

MySQL is used to create and manage the database. It stores student details, status updates, and history records. Proper relationships are defined to maintain data integrity. Tables are structured to avoid redundancy. Data is stored in an organized manner. It ensures quick retrieval and secure storage of information.

6. MODULE IMPLEMENTATION

The system is divided into modules such as Student Module, Admin Module, Status Management Module, and Data Management Module. Each module is developed and tested separately for better performance. This modular approach simplifies development. Each module performs specific functions. Modules are integrated after development. It improves system efficiency and maintainability.

7. SECURITY IMPLEMENTATION

Security measures are implemented to protect user data. Authentication, input validation, and session management are used to prevent unauthorized access and ensure safe data handling. It ensures system reliability. User credentials are verified securely. Data protection techniques are applied. It helps in preventing data misuse and cyber threats.

8. TESTING AND DEBUGGING

The system is tested at different stages to identify and fix errors. Various testing methods are used to ensure that all functionalities work correctly without issues. Debugging helps in improving system performance. Errors are detected and corrected effectively. System stability is ensured through testing. It guarantees smooth functioning of the application.

9. DEPLOYMENT AND MAINTENANCE

After successful testing, the system is deployed for use. Regular maintenance is carried out to ensure smooth performance and to update the system with new features if required. This phase ensures long-term usability. The system is monitored continuously after deployment. Updates are provided when needed. It ensures that the system remains efficient and up-to-date.

6.2 SAMPLE SCREEN CAPTURES

6.2.1 STUDENT LOGIN PAGE

St. John's College, Palayamkottai
Ex-Student Data Collection Portal

Home

Welcome Back!

Access your alumni profile, update your career status, and stay connected with your batchmates.

Student Login

Enter your Register Number and Name to access your dashboard

Register Number

Enter your Register Number

Full Name

Enter your name as per college records

Login

Fig 6.2.1 Student Login Page

6.2.2 STUDENT PROFILE UPDATE PAGE

St. John's College, Palayamkottai
Student Dashboard

Welcome, Saranya N

Logout

Profile Information

Full Name

Saranya N

Register ID

230812780212038

Email

saranya2005@gmail.com

Phone

6380454986

Current Status *

Working

Company Name *

ZOHO

Salary

40,000

ID Card or Offer Letter (Upload) *

Choose File No file chosen

Upload your ID card or offer letter as proof (Jpg, Png, or PDF)

Address

13/A Sankarankovil road Thuchanallur, Tirunelveli.

Upload Profile Photo (Jpg/Png)

Choose File No file chosen

Upload your profile photo

Signature (Upload)

Choose File No file chosen

Update Profile

Fig 6.2.2 Student Profile Update Page

6.2.3 ADMIN DASHBOARD PAGE

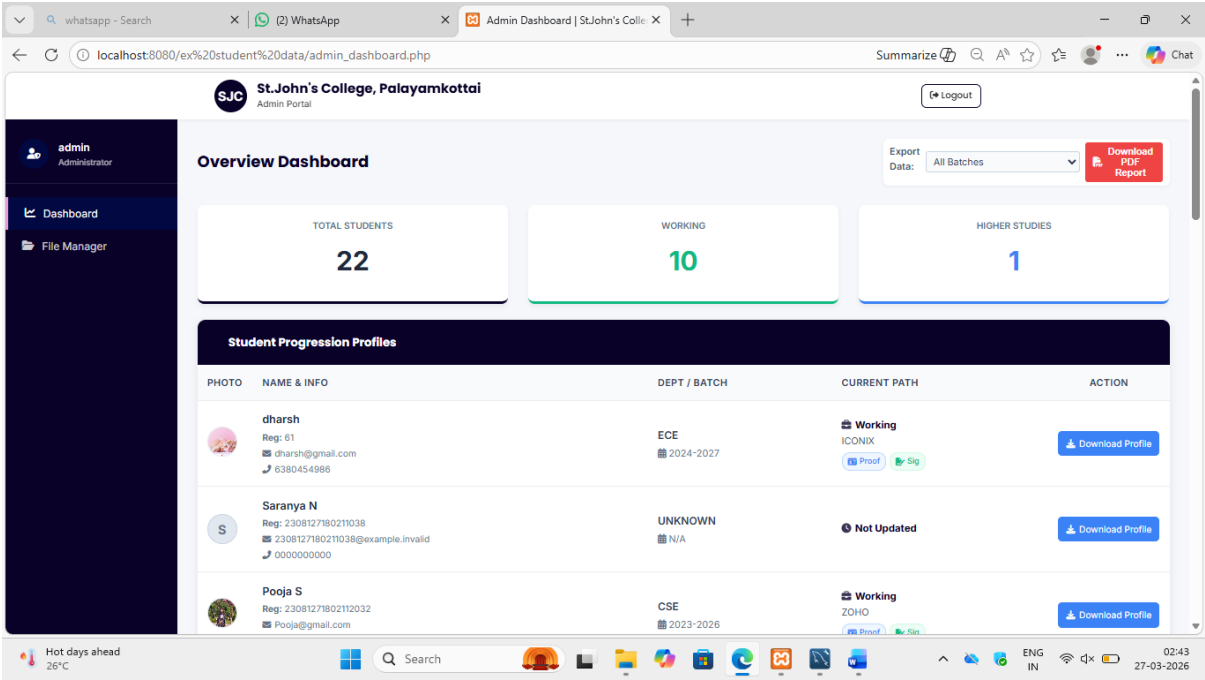


Fig 6.2.3 Admin Dashboard Page

6.2.4 STUDENT PROFILE MANAGER PAGE

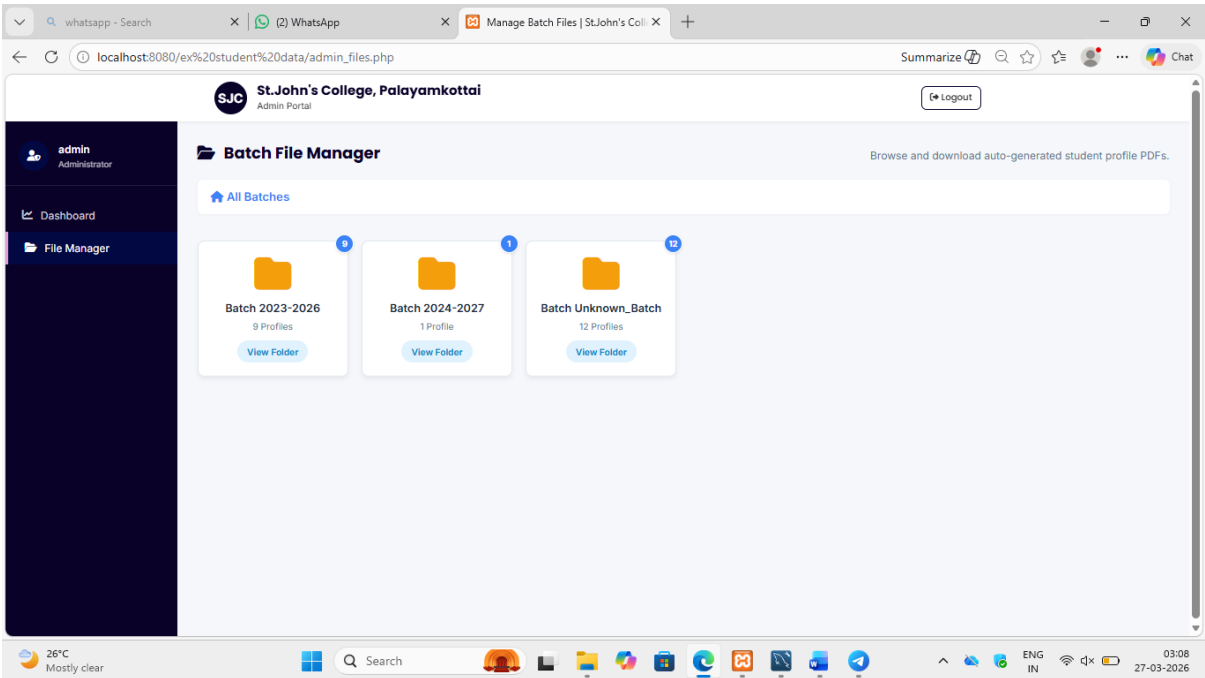


Fig 6.2.4 Student Profile Manager Page

6.2.5 STUDENTS FILE PAGE

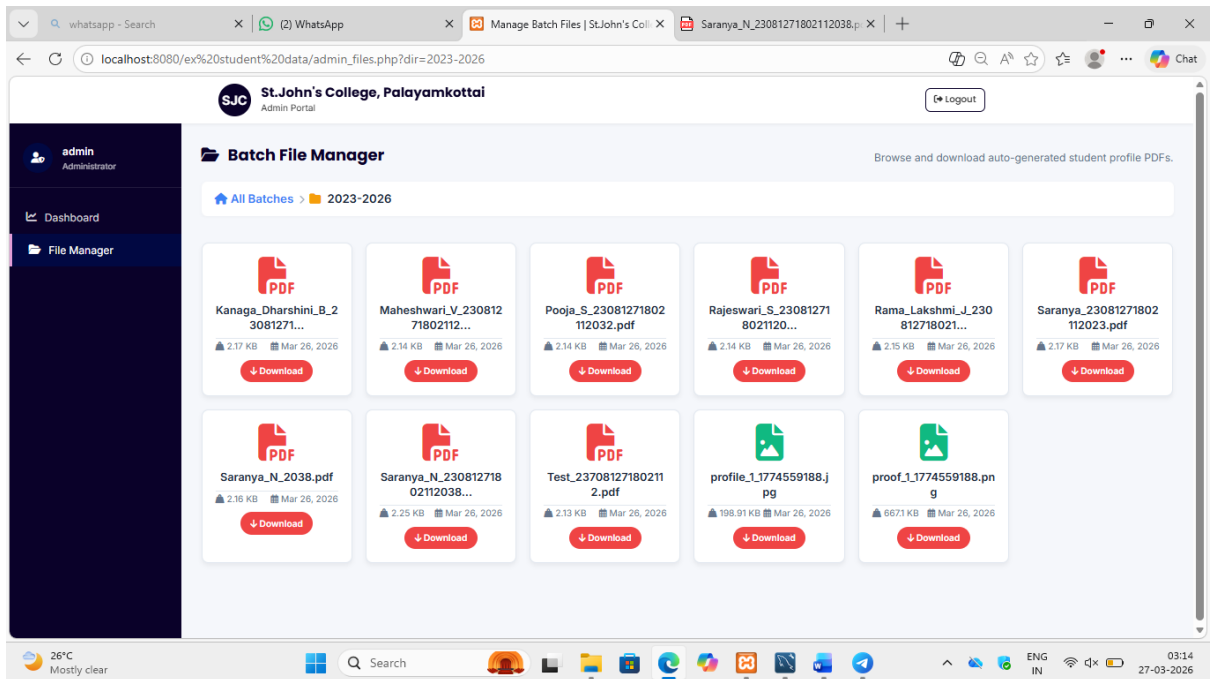


Fig 6.2.5 Students File Page

6.3 SAMPLE SOURCE CODE

Student_dashboard.html

```
<!DOCTYPE html>

<html lang="en"

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Student Dashboard | St.John's College</title>

<link rel="stylesheet" href="css/style.css">

<link          rel="stylesheet"          href="https://cdn.jsdelivr.net/npm/font-awesome/6.4.0/css/all.min.css">

</head>

<body>
```

```

<!-- Top Navigation -->

<nav class="navbar" style="padding: 10px 0;">

<div class="container navbar-container">

<div class="logo-container">

<a href="index.html" class="college-logo-placeholder" style="text-decoration:
none;">SJC</a>

<div class="college-name">

<h1>St.John's College, Palayamkottai</h1>

<span>Student Dashboard</span>

</div>

</div>

<div class="nav-links">

<span class="text-light" style="margin-right: 15px;">Welcome, User</span>

<a href="logout.php" class="btn btn-secondary" style="padding: 5px 15px; font-size:
0.85rem;">Logout</a>

</div>

<div class="mobile-menu-btn" id="sidebarToggle">≡</div>

</div>

</nav>

<div class="dashboard-container">

<!-- Sidebar -->

<aside class="sidebar">

<div class="user-profile-mini">

<div class="avatar-circle">JD</div>

<div>

```

```

<div style="font-weight: 600;">John Doe</div>

<div style="font-size: 0.8rem; opacity: 0.7;">CSE - 2024</div>

</div>

</div>

<div class="sidebar-menu">

<a href="#" class="active"><i class="fas fa-home"></i> Dashboard</a>

<a href="#"><i class="fas fa-user"></i> My Profile</a>

<a href="#"><i class="fas fa-briefcase"></i> Update Status</a>

<a href="#"><i class="fas fa-file-upload"></i> Upload Proof</a>

<a href="logout.php"><i class="fas fa-sign-out-alt"></i> Logout</a>

</div>

</aside>

<!-- Main Content -->

<main class="main-content">

<div class="page-header">

<h2>My Dashboard</h2>

<span class="badge badge-success">Verified Profile</span>

</div>

<div class="cards-grid mb-4">

<div class="info-card" style="border-top-color: var(--secondary-color);">

<h3>Current Status</h3>

<p class="mt-2 text-center"

style="font-size: 1.2rem; font-weight: bold; color: var(--primary-color);">Working

Professional

```

```

</p>

<p class="text-center text-light">Software Engineer at Tech Corp</p>

</div>

<div class="info-card" style="border-top-color: var(--warning);">

<h3>Verification Docs</h3>

<p      class="mt-2      text-center"><span      class="badge      badge-
success">Approved</span></p>

<p class="text-center text-light">Last updated: 10 Oct 2025</p>

</div>

<!-- Add more summary cards if needed -->

</div>

<!-- Profile Details Section -->

<div class="section-card">

<div class="card-header">

<h3>Profile Information</h3>

<button class="btn btn-accent" style="padding: 5px 15px; font-size: 0.8rem;">Edit
Profile</button>

</div>

<div class="card-body">

<div class="form-row">

<div class="mb-3">

<strong class="text-light">Full Name:</strong>

<div>John Doe</div>

</div>

<div class="mb-3">

```

<strong class="text-light">Register ID:

<div>2020CSE056</div>

</div>

<div class="mb-3">

<strong class="text-light">Department:

<div>Computer Science Engineering</div>

</div>

<div class="mb-3">

<strong class="text-light">Batch:

<div>2020 - 2024</div>

</div>

<div class="mb-3">

<strong class="text-light">Email:

<div>john.doe@example.com</div>

</div>

<div class="mb-3">

<strong class="text-light">Phone:

<div>+1 987 654 3210</div>

</div>

</main>

</div>

<script src="js/main.js"></script>

</body>

</html>

Admin_dashboard.php

```
<?php

session_start();

require_once 'db_connect.php';

// Check if admin is logged in

if(!isset($_SESSION['admin_logged_in']) || $_SESSION['admin_logged_in'] !== true)
{

header("Location: admin_login.php");

exit();

}

$admin_username = $_SESSION['admin_username'] ?? 'Admin';

$admin_role = $_SESSION['admin_role'] ?? 'staff';

$admin_dept = $_SESSION['admin_dept'] ?? "";

// Fetch stats and students

$queryParam = "";

if ($admin_role === 'staff' && !empty($admin_dept)) {

$stats = $conn->query("SELECT status_type, COUNT(*) as count FROM students
WHERE    department    =    '$admin_dept'    GROUP    BY    status_type")-
>fetch_all(MYSQLI_ASSOC);

$total = $conn->query("SELECT COUNT(*) as count FROM students WHERE
department = '$admin_dept'")->fetch_assoc()['count'];

$stmt = $conn->prepare("SELECT * FROM students WHERE department = ? ORDER
BY id DESC");

$stmt->bind_param("s", $admin_dept);

$stmt->execute();
```

```

$result = $stmt->get_result();

$stmt->close();

} else {

    $stats = $conn->query("SELECT status_type, COUNT(*) as count FROM students
    GROUP BY status_type")->fetch_all(MYSQLI_ASSOC);

    $total = $conn->query("SELECT COUNT(*) as count FROM students")->
    fetch_assoc()['count'];

    $sql = "SELECT * FROM students ORDER BY id DESC";

    $result = $conn->query($sql);

}

$working = 0;

$higher_studies = 0;

foreach($stats as $s) {

    if(strtolower($s['status_type']) == 'working') $working = $s['count'];

    if(strtolower($s['status_type']) == 'higher studies') $higher_studies = $s['count'];

}

function getDriveDirectLink($url) {

    if (empty($url)) return "";

    if (strpos($url, 'drive.google.com') !== false) {

        $fileId = "";

        if (preg_match('/^d\/([a-zA-Z0-9_-]+)/', $url, $matches)) {

            $fileId = $matches[1];

        } elseif (preg_match('/id=([a-zA-Z0-9_-]+)/', $url, $matches)) {

            $fileId = $matches[1];

        }

    }

}

```



```

if ($fileId) return "https://drive.google.com/thumbnail?id=" . $fileId . "&sz=w300";

}

return $url;

}

?>

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title><?php echo $admin_role === 'admin' ? 'Admin' : 'Staff'; ?> Dashboard | St.John's
College</title>

<link rel="stylesheet" href="assets/css/style.css?v=<?php echo time(); ?>">

<link          rel="stylesheet"          href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.4.0/css/all.min.css">

<style>

.table-responsive { overflow-x: auto; background: white; border-radius: 10px; box-
shadow: 0 4px 6px rgba(0,0,0,0.05); }

.data-table { width: 100%; border-collapse: collapse; min-width: 800px; }

.data-table th, .data-table td { padding: 15px; text-align: left; border-bottom: 1px solid
#eee; vertical-align: middle; }

.data-table th { background-color: #f8f9fa; color: #343a40; font-weight: 600; font-size:
0.9rem; text-transform: uppercase; letter-spacing: 0.5px; white-space: nowrap; }

.data-table tr:hover { background-color: #f2f3f4; }

```

```
.stat-card { background: white; padding: 20px; border-radius: 10px; box-shadow: 0 4px 6px rgba(0,0,0,0.05); text-align: center; border-bottom: 4px solid var(--primary-color); }
```

```
.stat-card.success { border-bottom-color: #10b981; }
```

```
.stat-card.info { border-bottom-color: #3b82f6; }
```

```
.stat-number { font-size: 2.5rem; font-weight: bold; margin: 10px 0; color: #1e293b; }
```

```
.stat-label { color: #64748b; font-weight: 600; text-transform: uppercase; font-size: 0.85rem; }
```

```
.student-photo { width: 45px; height: 45px; border-radius: 50%; object-fit: cover; border: 2px solid #e2e8f0; cursor: pointer; transition: transform 0.2s; }
```

```
.student-photo:hover { transform: scale(1.1); }
```

```
.photo-placeholder { width: 45px; height: 45px; border-radius: 50%; background: #e2e8f0; display: flex; align-items: center; justify-content: center; \
```

```
</div>
```

```
</div>
```

```
<div class="nav-links">
```

```
<a href="admin_logout.php" class="btn btn-secondary btn-sm"><i class="fas fa-sign-out-alt"></i> Logout</a>
```

```
</div>
```

```
<div class="mobile-menu-btn" id="sidebarToggle"><i class="fas fa-bars"></i></div>
```

```
</div>
```

```
</nav>
```

```
<div class="dashboard-container">
```

```
<!-- Sidebar -->
```

```
<aside class="sidebar">
```

```
<div class="user-profile-mini">
```

```

<div class="avatar-circle" style="background-color: var(--secondary-color); color:
white;"><i class="fas fa-user-shield"></i></div>

<div>

<div style="font-weight: 600;"><?php echo htmlspecialchars($admin_username);
?></div>

<div style="font-size: 0.8rem; opacity: 0.7;">

<?php echo $admin_role === 'admin' ? 'Administrator' : 'Staff - ' . ($admin_dept ? 'All
Depts'); ?>

</td>

<td style="text-align:center;">

<a href="download_student.php?id=<?php echo $row['id']; ?>" target="_blank"
class="btn-download" title="Download Print Profile">

<i class="fas fa-download"></i> Download Profile

</a>

</td>

</tr>

<?php endwhile; ?>

<?php else: ?>

<tr>

<td colspan="5" style="text-align: center; padding: 50px 20px;">

<i class="fas fa-folder-open" style="font-size: 3rem; color: #cbd5e1; margin-
bottom:15px; display:block;"></i>

<div style="color: #64748b; font-size:1.1rem; font-weight:500;">No student records
found.</div>

<p style="color:#94a3b8; font-size:0.9rem; margin-top:5px;">Check back later when
students update their complete profiles.</p>

```

```
</td>

</tr>

<?php endif; ?>

</tbody>

</table>

</div>

</div>

</div>

</main>

</div>

<!-- Script imports -->

<script src="assets/js/script.js"></script>

<script>

document.getElementById('sidebarToggle')?.addEventListener('click', function() {

document.querySelector('.sidebar').classList.toggle('active');

});

</script>

</body>

</html>
```

CHAPTER 7

CONCLUSION

The development and implementation of the **Student Progression Management System** marks the successful completion of a digital solution aimed at improving the management and tracking of student progression data. The system effectively addresses the limitations of traditional methods by providing a centralized, structured, and efficient platform for both students and administrators.

The application has been developed using a reliable and efficient technology stack, including **HTML, CSS, and JavaScript** for the frontend, along with PHP and MySQL for backend processing and database management. This ensures that the system is scalable, secure, and capable of handling multiple users and data updates efficiently.

The system successfully achieves its objectives by enabling students to update their personal and professional details, including their current status such as working, higher studies, business, or others. It also provides administrators with effective tools to monitor student data, analyse progression, and generate reports. The inclusion of status tracking, document management, and history maintenance improves data organization and accessibility.

In conclusion, the project has been successfully designed and implemented as a functional and efficient system that meets the requirements of modern educational institutions. **The Student Progression Management System** simplifies data management, improves efficiency, and serves as a reliable platform for maintaining structured student records, with the potential for future enhancements.

CHAPTER 8

FUTURE ENHANCEMENT

The Student Progression Management System can be further improved by incorporating additional features and advanced technologies to enhance its performance, usability, and scalability.

The following enhancements can be implemented in the future:

1. **Mobile Application Development:** The system can be extended to a mobile application, allowing students to update their details easily through smartphones. This will increase accessibility and user convenience.
2. **Notification System:** Email and SMS notification features can be added to remind students to update their status periodically. This ensures that the database remains up-to-date at all times.
3. **Advanced Data Analytics:** The system can include analytical tools to generate reports based on student career trends.
4. **Enhanced Security Features:** Advanced security measures such as OTP verification, data encryption, and secure authentication can be implemented to protect sensitive student information.
5. **Cloud Integration:** The system can be integrated with cloud storage to improve scalability, data backup, and accessibility. This will allow handling of large volumes of data efficiently.
6. **Data Visualization and Analytics:** Graphs and charts can be used to show student details in a clear way. It helps the admin easily understand how many students are working, studying, or doing business, and makes it easier to analyze and make decisions.
7. **Automated Verification:** Automatic verification of uploaded documents can be introduced to improve accuracy and reduce manual checking by administrators.
8. **Role-Based Access:** Different roles like staff, admin, and HOD can be added in the system. Each user will have specific access permissions. This improves system management and security.

9. Security Enhancements: Advanced security features such as data encryption, OTP login, and secure authentication can be implemented to protect user data.

10. Integration with Placement Systems: The system can be integrated with college placement systems to track job placements and career growth of students.

These future enhancements will make the Student Progression Management System more powerful, efficient, and adaptable to modern requirements. By implementing these features, the system can provide better performance and support large-scale usage in educational institutions.

CHAPTER 9

BIBLIOGRAPHY

The **bibliography** section lists all the sources of information and references that were used during the development of the **Student Progression Management System**. It includes **books**, **websites**, and online resources that helped in understanding the concepts of web development, database management, and system design.

HTML & CSS: Design and Build Websites

- Jon Duckett, Wiley Publications

Learning PHP, MySQL & JavaScript

- Robin Nixon, O'Reilly Media

Web Database Applications with PHP and MySQL

- Hugh E. Williams & David Lane

MySQL Official Documentation

- Available at: <https://dev.mysql.com/doc/>

PHP Official Documentation

- Available at: <https://www.php.net/docs.php>

W3Schools – Web Development Tutorials

- Available at: <https://www.w3schools.com/>

MDN Web Docs (Mozilla Developer Network)

- Available at: <https://developer.mozilla.org/>

Stack Overflow – Programming Help Community

- Available at: <https://stackoverflow.com/>

Geeks for Geeks – Programming and Database Concepts

- Available at: <https://www.geeksforgee>